ELEC50010 Instruction Architectures and Compilers
May 2026

NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–9 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.

1. Using the instruction summary provided in the Appendix, translate the following RISC-V RV32I instructions either from assembly language to machine code or vice versa.

   a) `sw t0, 256(s10)` [3]

   Solution. *This question requires students to show that they understand how RISC-V instructions are encoded. 16'h105d2023*

   b) 32'h02090493 [3]

   Solution. *addi s1, s2, 0x20*

2. The current values of the RISC-V processor's programme counter PC and register s1 are respectively:

   PC = 32'h1000; s1 = 32'h400

   What is the value of the register s0 after the following individual single instruction is executed separately:

   a) `auipc s0, 20'hA0001` [4]

   Solution. *This question tests student's understanding of unsigned immediate numbers and program counter relative instructions.*

   s0 = 32'hA0001000 + 32'h00001000 = 32'hA0002000.

   b) `slti s0, s1, 12'h800` [4]

   Solution. *s1 = 32'h400.*

   Immediate value must be signed extended, with zero filling the other bits to 32'hfffff800, a negative number.

   Therefore s1 is larger than the immediate value after sign extension.

   Hence, s0 = 32'h0, i.e. NOT set.

3. The Branch (B-type) and Jump (J-type) instruction formats are shown in Figure 3.1.

   a) Briefly describe how the immediate value is computed by the assembler for all the branch instructions? [4]

   Solution. *This question tests student's understanding of encoding of the*

Answers

immediate values in Branch and Jump instructions, the working of the JALR instruction, and how to design the immediate operand module for these two instruction types in SystemVerilog.

The branch instruction includes a label, which is the Branch Target Address (BTA). The 32-bit offset immediate operand is calculated as offset = BTA – instruction's PC value as a 2's complement number. The instruction stores only stores bit 12 to bit 1 of the offset value.

b) Briefly describe the operation of the instruction jalr t0, t1, 31. [4]

Solution. *The jalr t0, t1, 31 instruction performs the following operation:*

1. T0 is loaded with PC + 4.
2. Bit 12 to bit 31 of the instruction is used to form a 32-bit signed offset, which is added to t1 to form the destination address.
3. This value is loaded to the PC, effecting a jump to this destination address.

c) Design in SystemVerilog the module *imm ()* as shown in Figure 3.2, which takes the 32-bit instruction code for B-type and J-type instructions and produces as output the 32-bit immediate operand required depending on the value of the input control signal B-type (true for Branch instructions). [6]

Solution.

```
module imm(
    input logic [31:0]  instr,      // 32-bit machine code instruction
    input logic         B_type,     // 1 for B-type, 0 for J-type
    output logic [31:0] imm_op      // 32-bit immediate operand
);

    assign imm_op = (B_type ?
                       {20{instr[31]}, instr[7], instr[30:25], instr[11:8], 1'b0}
                     : {12 instr[31]}, instr[19:12], instr[20], instr[30:21], 1'b0};
    );
endmodule
```

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]											[11]	imm[19:12]							rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4

Figure 3.1

```
module imm(
    input logic [31:0]  instr,      // 32-bit machine code instruction
    input logic         B_type,     // 1 for B-type, 0 for J-type
    output logic [31:0] imm_op      // 32-bit immediate operand
);
```

Figure 3.2

Answers

4. Figure 4.1 shows the state diagram of a finite state machine (FSM) that implements the 4-state branch prediction algorithm. Design in SystemVerilog the FSM using the module interface shown in Figure 4.2. [12]

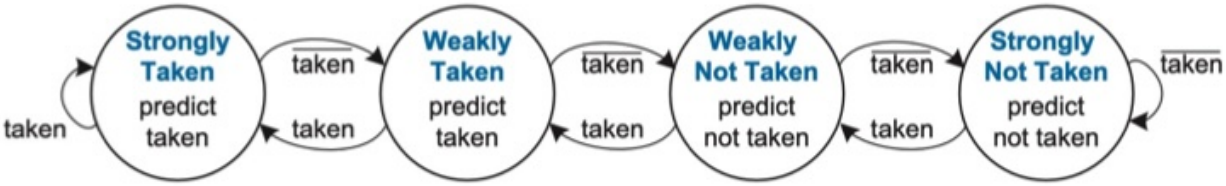


Figure 4.1

```
module branch_prediction_FSM(
    input  logic clk,            // system clock signal
    input  logic reset,          // FSM resert
    input  logic branch_taken    // branch resolved as taken
    output logic predict_taken   // 32-bit immediate operand
);
```

Figure 4.2

Solution. *This question tests student's ability to implement a FSM in SystemVerilog.*

Answers



```
module branch_prediction_FSM(
    input  logic clk,            // system cl
    input  logic reset,          // FSM reser
    input  logic branch_taken    // branch re
    output logic predict_taken   // 32-bit im
);

    // define states
    typedef enum {ST, WT, WNT, SNT} my_state;
    my_state   current_state, next_state;

    // state registers
    always_ff @(posedge clk)
        if (reset)  current_state <= SNT;
        else        current_state <= next_state;

    // next state logic
    always_comb
        case (current_state)
            ST: if (branch_taken==1'b0) next_state = WT;
                else next_state = ST;
            WT: if (branch_taken==1'b0) next_state = WNT;
                else next_state = ST;
            WNT: if (branch_taken==1'b0) next_state = SNT;
                else next_state = WT;
            SNT: if (branch_taken==1'b0) next_state = SNT;
                else next_state = WNT;
        default: next_state = current_state;
        endcase

    // output logic
    always_comb
        case (current_state)
            ST: predict_taken = 1'b1;
            WT: predict_taken = 1'b1;
            WNT: predict_taken = 1'b0;
            SNT: predict_taken = 1'b0;
        default: predict_taken = 1'b0;
        endcase
endmodule
```

Arm Confidential

5. a) Explain briefly how principles of temporal and spatial locality enable cache memories to speed up the execution of software in computers. [4]

Solution. *This question tests student's understanding of caches.*

The basis of cache memory is that memory "furthest" from the CPU takes longer to access. Therefore if we provide small amount of fast closer to the CPU, we can improve the processor performance because of:

1. Temporal Locality – any instruction or data used previously is more likely to be used again. In the case of instruction memory, programs are often executing tight loops. That means recently executed instructions are likely to be executed again.

2. Spatial Locality – any instruction or data accesses are likely to be close to each other in address space. Instructions are executed in sequence unless that is a jump or branch. Therefore, accessing instructions are likely to be spatially close. Data accesses are often to arrays or other data structures that are in consecutive address locations.

b) A processor with data cache is used to execute a program that has 10,000 loads operations from data memory. 8123 data access can be found in the cache memory and the rest are in other level of memory hierarchy. Calculate the cache hit and miss rates. Given that cache access takes 1 clock cycle while all other memory accesses require 100 clock cycles, calculate the total expected memory access time of the program. [6]

Solution.

Hit rate = 8123/10000 = 81.23

Average memory access time (AMAT) = $t_{cache} + MR_{cache} \times t_{MM}$

$t_{cache} = 1$, $t_{MM} = 100$.

AMAT = (1 + 0.1877 x 100) x 10,000 = 187,700 clock cycles.

Answers

6. Explain how `continue` statements are handled by a C compiler. *[Hint: Note that* `continue` *statements may be compiled differently depending on where they appear in the source code.]* [6]

Solution. *The* `continue` *statement is compiled to an unconditional jump (1 mark). The target of the jump is a label that is at the beginning of (1 mark) the body of the innermost (1 mark) loop (1 mark) that contains the* `continue`*. If the* `continue` *statement is not inside a loop statement, then this is a programmer fault, and the compiler should generate an error message (2 marks).*

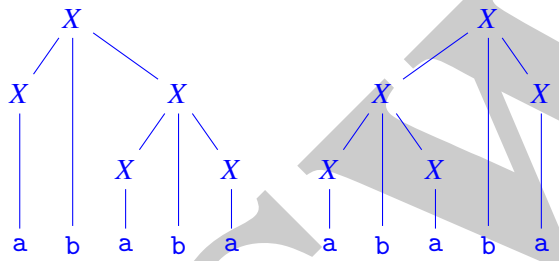
Comment. *Students were expected to demonstrate understanding of* `continue` *statements from the point of view of the* compiler writer*, not simply from the point of view of the* programmer*. So it was not sufficient to simply describe the* behaviour *of the* `continue` *statement – I needed to see that you understood how the statement is* implemented*. Note that* `continue` *statements, in contrast to* `break` *statements, are* not *allowed inside switch or if-statements (unless they are also inside a loop).*

7. Here is an *ambiguous* context-free grammar:

$$X ::= \texttt{a} \mid X\,\texttt{b}\,X$$

a) Demonstrate that this grammar is ambiguous by drawing two different parse trees for the word `ababa`. [5]

Solution. *Here are two parse trees:*



b) Is X *nullable*? Briefly explain your answer. [2]

Solution. *No, because it cannot generate the empty word.*

c) What is the *FIRST* set of X? Briefly explain your answer. [2]

Solution. $\{\texttt{a}\}$*, because every word generated by X begins with an* `a`*.*

d) What is the *FOLLOW* set of X? Briefly explain your answer. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [3]

Solution. $\{\texttt{b}, \$\}$*, because every word generated by X is followed either by a* `b` *or the end of the input.*

e) Write down another context-free grammar that accepts the *same language*, but is no longer ambiguous. [4]

Solution. *The simplest option is* $X ::= \texttt{a} \mid \texttt{ab}X$*.*

Comment. *Some students provided an equivalent answer in the form of a regular expression rather than a context-free grammar (e.g.* $\texttt{a}(\texttt{ab})^*$*) and I gave up to 2 marks for this.*

8. Gregory wants to make use of some library code in his program. He must decide whether to *statically* or to *dynamically* link his program with the library.

a) Explain one mechanism by which statically linking the library could make Gre-

Answers

gory's program *more likely to crash*. [2]

Solution. *Gregory's program cannot benefit from bugfixes made to the library after compilation.*

Comment. *Many students wrote that the statically linked executable, being larger, is more likely to cause a crash simply by causing the computer to run out of memory. I think this is a bit far-fetched, but I gave up to 1 mark for answers in this vein.*

b) Explain one mechanism by which statically linking the library could make Gregory's program *less likely to crash*. [2]

Solution. *Gregory's program is not reliant on finding (the right version of) the library at runtime.*

c) Explain one mechanism by which statically linking the library could make Gregory's program *faster to start up*. [2]

Solution. *No need for the operating system to locate the shared library and link it in during start-up.*

Comment. *Many students wrote that the statically linked executable has better spatial locality of reference because all its functions are close together in memory, and hence is more amenable to caching. I gave only 1 mark for such answers because this question is specifically referring to the impact on* start-up *time, not the performance of the program while it is running.*

d) Explain one mechanism by which statically linking the library could make Gregory's program *slower to start up*. [2]

Solution. *The shared library might already be in memory, having been loaded by an already-running program.*

9. Consider the following C function:

```
int foo (int a) {
  int b, c, d; // 1
  b = a + 4;   // 2
  c = b - a;   // 3
  d = b + c;   // 4
  a = b + c;   // 5
  return a;
}
```

a) Annotate `foo` with the set of variables that are *live* at the end of each line (the positions labelled `1`–`5` in the code). [5]

Solution.

```
int foo (int a) {
  int b, c, d; // {a}
  b = a + 4;   // {a,b}
  c = b - a;   // {b,c}
  d = b + c;   // {b,c}
  a = b + c;   // {a}
  return a;
}
```

b) Give two benefits of the *dead-code elimination* optimisation. [4]

Answers

Solution. *Any two of: (1) reduces code size, (2) faster execution, (3) less memory needed.*

c) Explain how the analysis carried out in part (a) enables dead-code elimination to be performed on `foo`. [2]

Solution. *The analysis shows that* `d` *is not live immediately after being assigned. This means that the assignment is dead and can be removed.*

d) Now optimise `foo` *as much as possible*, and write down 32-bit RISC-V assembly code that implements it. Explain why your assembly code correctly implements `foo`. [9]

Solution. *After optimisation,* `foo` *collapses to a function that simply adds* `8` *to its input. The input arrives in register* `a0` *and the return value goes in* `a0`*, so adding* `8` *to the contents of* `a0` *and then returning is all that is required.*

```
foo:
        addi    a0,a0,8
        ret
```

Comment. *Many students forgot the* `ret` *instruction. Some students included instructions for setting up and tearing down* `foo`*'s stack frame; these aren't necessary for an implementation that is optimised 'as much as possible' but I didn't penalise such answers (unless the extra instructions were wrong). Some students only optimised* `foo` *by removing the dead assignment to* `d`*, so they ended up with rather longer implementations than given in my model answer above, but again I didn't penalise these answers unless the extra instructions were wrong.*

Answers

RISC-V Instruction Set Summary (1)

Register (R-type) Instructions

Bit position	31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
R-type	funct7	rs2	rs1	funct3	rd	opcode

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
7'h33	3'b010	7'h00	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
7'h33	3'b011	7'h00	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)^{unsigned}
7'h33	3'b100	7'h00	xor rd, rs1, rs2	bit-wise XOR	rd = rs1 ^ rs2
7'h33	3'b101	7'h00	srl rd, rs1, rs2	shfit right logical	rd = rs1 >> $rs2_{4:0}$
7'h33	3'b101	7'h20	sra rd, rs1, rs2	shfit right arithmetic	rd = rs1 >>> $rs2_{4:0}$
7'h33	3'b110	7'h00	or rd, rs1, rs2	bit-wise OR	rd = rs1 | rs2
7'h33	3'b111	7'h00	and rd, rs1, rs2	bit-wise AND	rd = rs1 & rs2

Immediate (I-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
I-type	imm[11:0]	rs1	funct3	rd	opcode

opcode	funct3	imm[11:5]	instruction	description	operation
7'h03	3'b000	-	`lb rd, imm(rs1)`	load byte immediate	rd = sign_ext ($[address]_{7:0}$)
7'h03	3'b001	-	`lh rd, imm(rs1)`	load half immediate	rd = sign_ext ($[address]_{15:0}$)
7'h03	3'b010	-	`lw rd, imm(rs1)`	load word immediate	rd = $[address]_{31:0}$
7'h03	3'b100	-	`lbu rd, imm(rs1)`	load byte unsigned	rd = zerp_ext ($[address]_{7:0}$)
7'h03	3'b101	-	`lhu rd, imm(rs1)`	load half unsigned	rd = zero_ext ($[address]_{15:0}$)
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b001	7'h00	`slli rd, rs1, uimm`	Shift left logical immediate	rd = rs1 << uimm
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))
7'h13	3'b011	-	`sltiu rd, rs1, uimm`	set less than imm. unsigned	rd = (rs1 < zero_ext(imm))
7'h13	3'b100	-	`xori rd, rs1, imm`	bit-wise XOR immediate	rd = rs1 ^ sign_ext (imm)
7'h13	3'b101	7'h00	`srli rd, rs1, uimm`	Shift right logical immediate	rd = rs1 >> uimm
7'h13	3'b101	7'h20	`srai rd, rs1, uimm`	Shift right arithmetic imm.	rd = rs1 >> uimm
7'h13	3'b110	-	`ori rd, rs1, imm`	bit-wise OR immediate	rd = rs1 | sign_ext (imm)
7'h13	3'b111	-	`andi rd, rs1, imm`	bit-wise AND immediate	rd = rs1 & sign_ext (imm)

Upper Immediate (U-type) Instructions

Bit position	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12	11 10 9 8 7	6 5 4 3 2 1 0
U-type	imm[31:12]	rd	opcode

opcode	instruction	description	operation
7'h17	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
7'h37	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}

RISC-V Instruction Set Summary (2)

Store (S-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S-type	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						

opcode	funct3	instruction	description	operation
7'h23	3'b000	`sb rs2, imm(rs1)`	store byte immediate	$[address]_{7:0} = rs2_{7:0}$
7'h23	3'b001	`sh rs2, imm(rs1)`	store half immediate	$[address]_{15:0} = rs2_{15:0}$
7'h23	3'b010	`sw rs2, imm(rs1)`	store word immediate	$[address]_{31:0} = rs2$

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA
7'h63	3'b001	`bne rs1, rs2, label`	branch if not equal	If (rs1 ≠ rs2) PC = BTA
7'h63	3'b100	`blt rs1, rs2, label`	branch if less than	If (rs1 < rs2) PC = BTA
7'h63	3'b101	`bge rs1, rs2, label`	branch if greater than or equal	If (rs1 ≥ rs2) PC = BTA
7'h63	3'b110	`bltu rs1, rs2, label`	branch if less than unsigned	If (rs1 < rs2) PC = BTA
7'h63	3'b111	`bgeu rs1, rs2, label`	branch if greater or equal unsigned	If (rs1 ≥ rs2) PC = BTA

BTA = Branch Target Address, PC = Program Counter

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]										[11]	imm[19:12]								rd					opcode						
jalr – I-type	imm[11:0]												rs1					3'b000			rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4
7'h6f	`jal rd, label`	Jump and link	PC = JTA, rd = PC + 4

JTA = Jump Target Address, PC = Program Counter

Register names

name	register number	usage
zero	x0	Constant value 0
ra	x1	Return Addrss
sp	x2	Stack Pointer
gp	x3	Global Pointer
tp	X4	Thread Pointer
t0 – t2	x5 – x7	Temporary registers
s0/fp	x8	Saved register/Frame Pointer
s1	x9	Saved register
a0 – a1	x10 – x11	Function arguments/ return values
a2 – a7	x12 – x17	Function arguments
s2 – s11	x18 – x27	Saved registers
t3 – t6	x28 – x31	Temporary registers

Abbreviations

imm	12-bit signed immediate ($imm_{11:0}$)
uimm	5-bit unsigned immediate ($imm_{4:0}$)
upimm	20-bit upper immediate ($imm_{31:12}$)
address	memory addres: rs1 + sign_ext($imm_{11:0}$)
[address]	data at memory location address
BTA	branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0})
JTA	jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0})
label	text label indicating instruction address
sign_ext	value sign-extended to 32 bits
zero_ext	value zero-extended to 32 bits